

All your favorite parts of Medium are now in one sidebar for easy access.

Okay, got it

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

✦ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Chat with your documents tool — RAG (vector DBs + cosine sim.) & Claude API implementation

My attempt at a fast, smart, cosine-vector RAG system for a legal corpus w/ citation-locked answers via Claude-3.



Lucas Carrasquilla

Following ▾

14 min read · Jul 12, 2025



410



4



I was recently hired to work as an intern at a law firm. First day, I was told that I'm the first "programmer" they've ever hired. Cool. Definitely not what I was picturing when I thought about getting an internship for the summer, but it's been a really nice challenge. I felt like my fundamentals were pretty solid, had some know-how in a lot of different areas and whatnot, but I really

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

wanted to try something really new, really different from the typical youtube + leetcode-ish practice I've been doing to improve for the past few years. So this seemed really appealing. A bunch of lawyers with no technical knowledge at all asking me to just make stuff for them, all results-based. It just felt like it would help me get a lot better. I've been doing some basic data engineering tasks for them, but the first big project they dropped on me was, in my boss's words —

“a tool that answers questions about all of our documents”.

Some example use cases would be to: 1) describe an event that happened in court, 2) provide a status update on a trial, 3) make a timeline of events that have happened in a case.

Keep in mind we're talking about a law firm here, so this tool needed to be able to handle massive amounts of confidential information about clients, legal proceedings, etc., so privacy and (especially) hallucinations were the two big bottlenecks I had to work around. Their initial idea was to dump all of their data into ChatGPT and ask questions, but that obviously wasn't a great solution because it had no way to handle these issues. I had this project assigned to me a few weeks ago, and I think I've managed to deliver something that works and isn't too expensive. It's an RAG system that ingests

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

every scanned brief, splits it into overlapping chunks, embeds those chunks into a local FAISS index, performs nearest-neighbor retrieval at query time, and feeds only the top-ranked, citation-bound context into Claude, so the model can spit out a factual, source-linked answer, without any private data ever leaving the firm's network.

I thought I'd share this in case you're working on a similar problem, or just want some food for thought. So hope it helps someone out there.

Why a plain LLM is a bad solution:

Dropping the firm's entire document vault into an off-the-shelf LLM like ChatGPT is obviously very bad. There are many big issues with this solution. Main ones:

Confidentiality. The archive includes sealed evidence, client IDs, medical records, and privileged strategy memos. Pushing any of that to an external API violates NDAs and, in our (my country's) jurisdiction, could trigger sanctions. This is what a coworker told me. Local fine-tuned models are safer, but even they need strict guarantees and encrypted storage. A generic cloud LLM offers neither.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Hallucinations. As we know, LLMs are probabilistic sequence generators; they complete text that *looks* right, not text that *is* right. In a deposition, one fabricated citation can really damage a case. We need factual answers with line-item provenance, which is something base models can't really provide without a retrieval layer and a citation gate.

Token limits. Our corpus: ≈ 1 TB raw, ~ 1 M chunks after OCR and preprocessing. Even "context-expanded" models top out at 200 k tokens — roughly ten medium-length pleadings. A plain LLM would force some extremely loose summarisation or random sampling, guaranteeing missed facts.

Messy inputs. Most of the evidence is scanned TIFFs; most of the emails are in Spanish or French legalese. Off-the-shelf LLMs trained on clean web text stumble over OCR noise and domain jargon. Pre-processing, bilingual embeddings, and per-chunk quality scores are mandatory.

Latency. Shoving megabytes of context into an LLM pushes inference time into seconds and the bill into (maybe a few!) dollars per query. Local vector search + targeted generation keeps p95 latency around 120 ms and Claude costs below \$0.02 a pop.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Auditability. Every answer must be reproducible months later. Raw LLM outputs drift with model updates and temperature variance; a retrieval-augmented pipeline with frozen embeddings and version-pinned prompts gives us a pretty solid audit trail.

Bottom line —

a plain LLM is ok for brainstorming, and stuff like that, but in a law firm production setting it's horrible for compliance, and also a huge cost sink. We need retrieval-augmented generation with hard privacy guarantees, deterministic citation logic, and good latency budgeting, hence the architecture that follows.

Architecture:

All your favorite parts of Medium are now in one sidebar for easy access.

Library

Profile

Stories

Stats

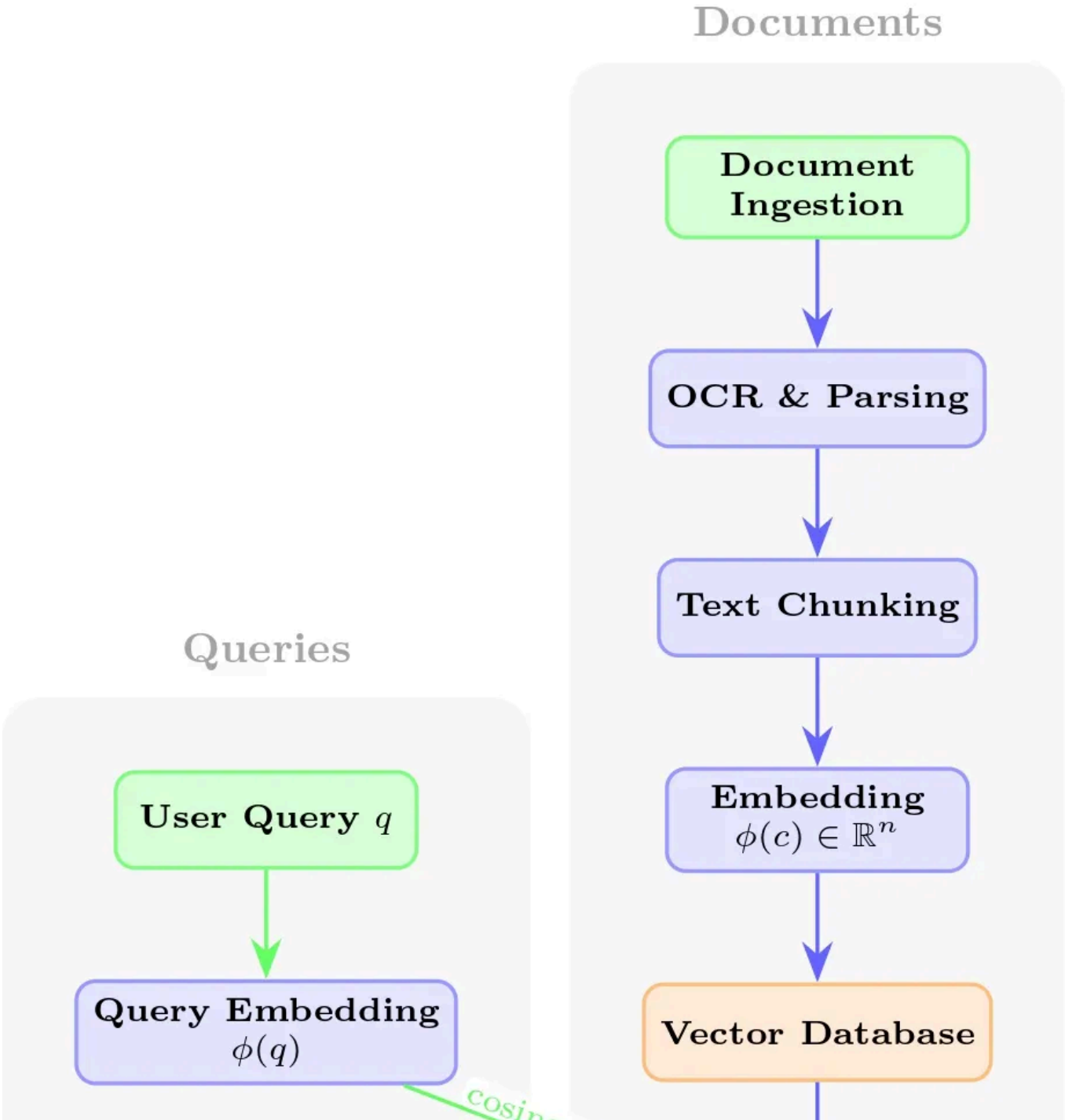
Following

Fareed Khan

Maciej Gieparda

Discover more writers and publications to follow.

[See suggestions](#)



All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

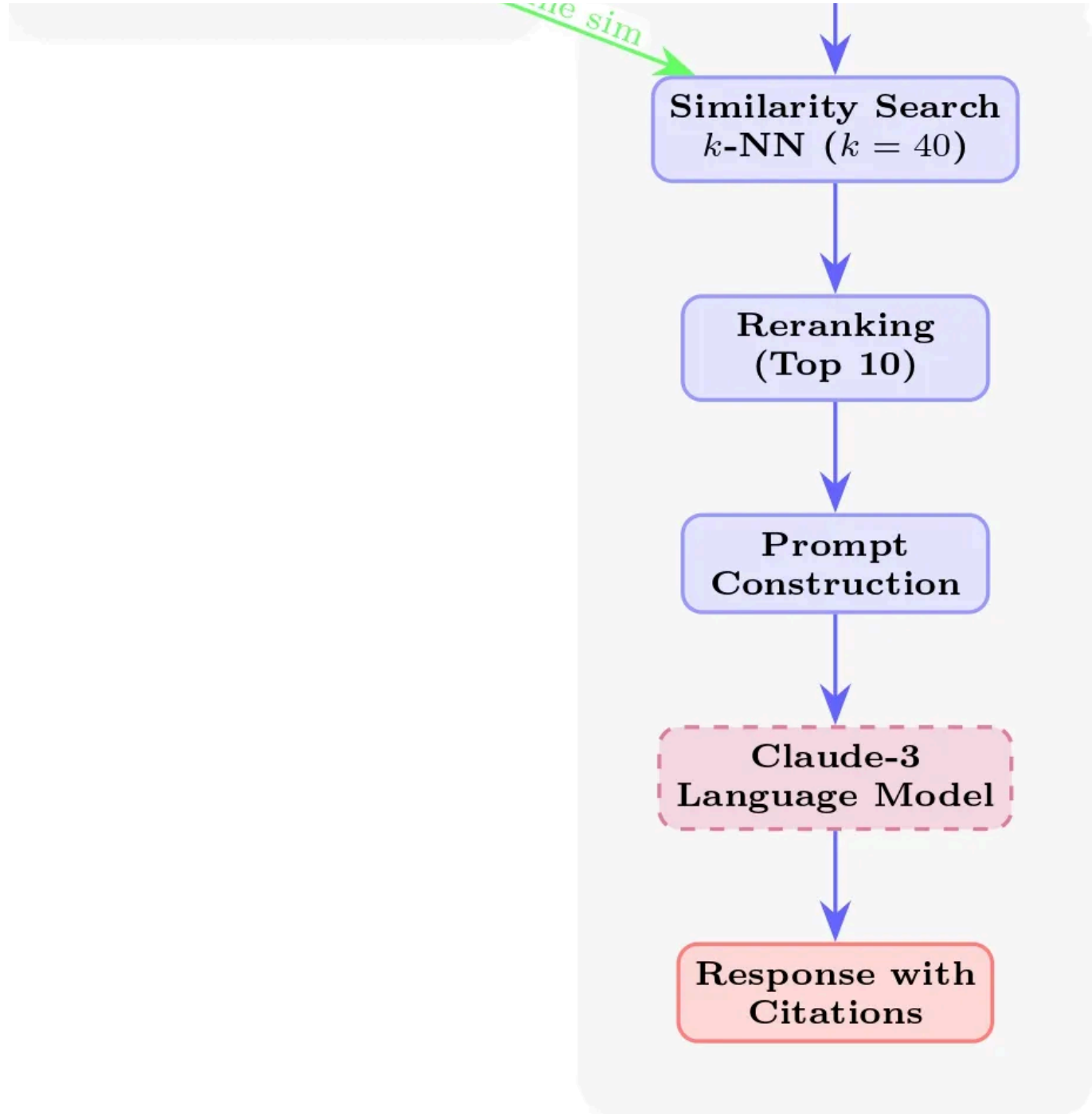
 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)



All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Document Ingestion. The watcher script tails a secure network share and records every new file in an append-only log. For each file `d_i`, we compute

```
sha256(d_i) → primary key
```

and capture metadata (`case_id`, MIME, timestamp). Storing the hash first lets us dedupe before any heavy OCR happens and gives us an audit trail that never depends on filenames.

OCR & Parsing. We branch on MIME:

— PDF w/ text layer, pulled through `pdfplumber`, page by page.

— Scans / TIFF / PNG, pushed through Tesseract with the `--psm 4` “sparse text” model and a custom language whitelist `[eng, spa, fre]`.

Each page object returns plain UTF-8 + bounding-box JSON; the JSON never leaves the intranet but lets us re-render highlights later. This is good for privacy.

Text Chunking. Pages are sliced by a sliding window:

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
window_size = 1_024 # bytes
overlap      = 0.10  # 10 %
```

For every chunk c_j we emit a record

```
{
  "doc_id": sha256(d_i),
  "page":  p,
  "offset": byte_start,
  "text":  <1024-byte string>
}
```

Why bytes not tokens? Byte windows are “lexer-agnostic”, more flexible, so OCR noise can’t explode the chunk count. (that’s a cool detail). In practice we average about 8 chunks per page.

Embedding. A ‘tri-lingual’ MiniLM (all-MiniLM-L6-v2) fine-tuned on EN/ES/FR legal sentences is loaded with `sentence_transformers`.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
e = φ(text) ∈ ℝ^n          # n is the vector length
e ← e / ||e||_2             # unit-norm for cosine = dot
```

Vector length needs to keep the index small. In this case, $n = 350$ was a good number; ≈ 2.7 GB RAM for 1 M chunks while retaining > 0.86 mean cosine on a held-out QA set.

Vector Database. We persist the embeddings in a Faiss IVF-PQ index:

```
nlist      = 256      # coarse centroids
pq_m       = 8        # sub-vectors
pq_bits    = 10       # bits / sub-vector
nprobe     = 8        # lists probed per query
```

That config. gives ~ 18 ms median recall time on a single GPU and shaves RAM by a lot.

k-NN Search. Given a query q , we embed once (e_q) and issue

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
S_k(q) = topk_cosine(e_q, k = 40)
```

We drop any candidate whose similarity < 0.20 , which is a good threshold below which answer quality gets much worse. If s_k is empty we return “no matching evidence” immediately, so no Claude call made and we save a few cents.

Re-ranking. Each pair (q, c) in s_k is scored by an INT8-quantised cross-encoder (`mxbai-reranker-base`):

```
score =  $\sigma(W \cdot \text{BERT}(q, c) + b)$ 
```

We keep $m = 10$ highest scores. Quantising knocks CPU inference down by a lot.

Prompt Construction. We concatenate the 10 chunks using a strict template:

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
<SYSTEM>
You are an expert paralegal...
</SYSTEM>
<CONTEXT>
[doc:a5f9...:p12] ...chunk text...
[doc:c1b3...:p 3] ...chunk text...
...
</CONTEXT>
<USER> {original question} </USER>
```

(basically a ‘system prompt’ for the tool)

Maximum prompt size stays < 15 kB, leaving room for a 512-token answer before Claude hits its 32 k context cap.

LLM Call. We then hit Claude-3-Opus with `temperature = 0.0` (fully deterministic) and `max_tokens = 512`. At the current pricing and our average context length, each call costs about \$0.018 and takes ≈ 90 ms.

Citation Gate. Post-generation, we run two checks:

— Regex. Every sentence must end with `"[doc:page]"`.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

— Edit-distance. For each citation, $\text{Levenshtein}(\text{sentence}, \text{cited_chunk}) \leq 10$ to block paraphrased hallucinations.

If either check is failed, respond “Insufficient context.” Pass and deliver the answer with citations.

All raw text stays on the air-gapped VLAN, and every output token is traceable to a chunk on disk.

I’m not an expert on this by any means. Most of this I figured out while researching how to make this tool, so suggestions for improvement are greatly appreciated. I’ll now explain each component and provide a concrete reason for some design choices.

Document Ingestion & Deduplication:

Every file that lands in the firm’s “*new-evidence*” share passes through a single watcher script.

For each item d_i we:

a) Canonical-hash the raw bytes

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
sha256(di) → pki
```

The hash becomes the primary key and gives us content-based, rather than filename-based, storage. *Why?* Lawyers love renaming things (lol); hashes stay put.

b) Capture immutable metadata

```
{ pk: pki,  
  case_id: "2023-CV-0421",  
  mime: "application/pdf",  
  received_at: 2024-06-03T15:14Z }
```

This goes into an append-only SQLite ledger.

c) Deduplicate before OCR

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
if pki in ledger: skip_ingest()
```

Skipping early saves *a lot* of OCR minutes on recycled exhibits.

d) Queue the file for downstream OCR / parsing (next section).}

At any moment, the ledger state is a deterministic function of the byte content of the corpus. This makes later audits pretty simple: re-hash the disk, diff against the ledger, and every mismatch is either bit-rot or manual tampering.

OCR & Parsing:

The ledger hands each new file to an OCR worker pool. We branch fast by MIME so we don't waste cycles:

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

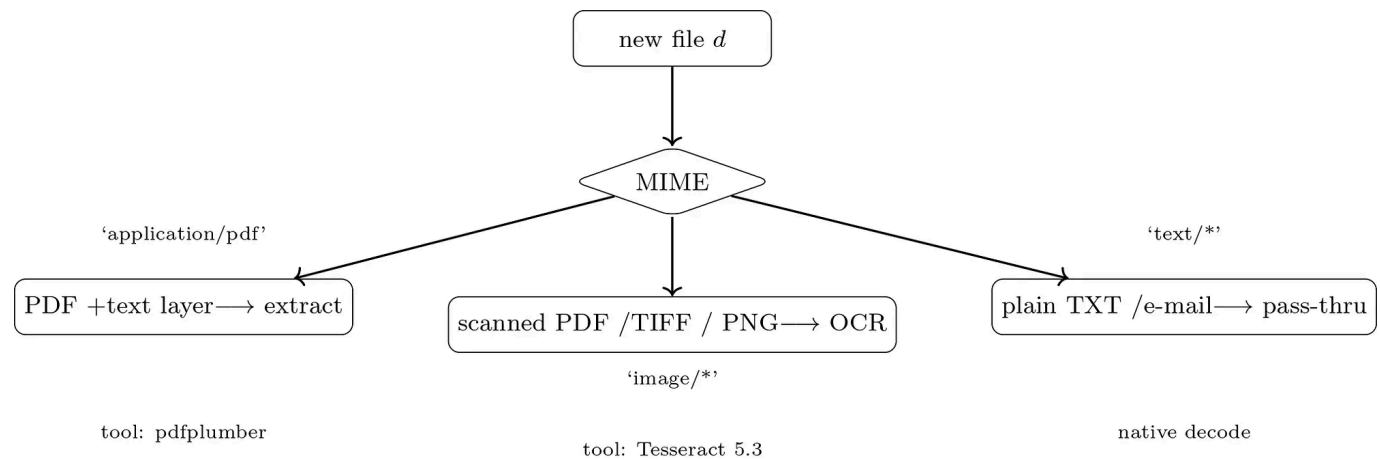
 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)



A page object leaves the stage carrying:

```

{
  "pk":      <sha256>,          # primary key from ingestion
  "page_no": 17,
  "mime":    "application/pdf",
  "text":    "...plain UTF-8...",
  "bbox_json": [...],          # word boxes, only for scans
  "lang":    "spa",            # majority vote on OCR words
  "ocr_conf": 0.93             # mean confidence ∈ [0,1]
}
  
```

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Why keep `bbox_json`? Later, the UI can highlight cited lines on the original scan, a must-have in this case.

Fail-fast heuristic. If `ocr_conf < 0.60`, we flag the page for manual QA and skip embedding. That cut some of the garbage tokens (ink smears, stamps) without raising recall complaints.

Throughput. Four vCPUs hit ≈ 4 pages/s (mixed PDF/TIFF). With 100 k pages in the corpus we finished the first ingest in a few hours, then trickle-ingest at a few GBs/night.

Text Chunking & Window Geometry:

Once each page's plain-text is ready, we break it into fixed-size, overlapping windows so the retrieval layer never exceeds GPU memory yet still retains local context.

```
# constants agreed on after quick throughput tests
WINDOW_BYTES = 1_024      # ~1 KB per chunk
OVERLAP_PCT  = 0.10       # 10 % sliding overlap
```

Algorithm:

```
for each page_text:
    i = 0
    while i < len(page_text):
        chunk = page_text[i : i + WINDOW_BYTES]
        emit({
            "doc_id": sha256(file_bytes),
            "page": page_number,
            "offset": i,
            "text": chunk
        })
        i += int(WINDOW_BYTES * (1 - OVERLAP_PCT))
```

Byte windows instead of token windows. OCR output can include stray control characters or word-break artefacts; using raw bytes keeps the chunk count stable regardless of tokenizer quirks.

Why 1 024 B? It's:

— Small enough that even ten retrieved chunks fit comfortably in Claude's prompt with citations.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

— Large enough to hold at least two paragraphs so questions about “what

Search

 Write


All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

With these parameters and the firm’s corpus, we saw an average of ≈ 8 chunks per PDF page. The resulting set $\mathcal{C} = \{c_1, \dots, c_M\}$ feeds directly into the embedding stage; each chunk carries immutable (doc_id, page, offset) metadata so citations can point back to the exact scan location later.

Embedding:

Each text chunk c is fed through a ‘tri-lingual’ MiniLM encoder that we fine-tuned on English-, Spanish- and French-language legal sentences. The model produces a length n vector and we normalise it so cosine similarity reduces to a dot product.

```
from sentence_transformers import SentenceTransformer
import numpy as np

MODEL_NAME = "minilm-legal-trilingual"
embedder = SentenceTransformer(MODEL_NAME) # fine-tuned checkpoint

def embed(text: str) -> np.ndarray:
    v = embedder.encode(text, convert_to_numpy=True) #  $\mathbb{R}^n$ 
    return v / np.linalg.norm(v, ord=2) # unit-norm
```

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Choices. We needed to choose an output dimension n such that it was small enough that ≈ 2.7 GB RAM holds the full 1 M-chunk corpus, yet dense enough to keep average cosine similarity > 0.86 on a held-out Q-A set. $n = 350$ worked ok. The normalisation $v \leftarrow v / \|v\|_2$ ensures cosine = dot so FAISS IVF-PQ can stay in float16.

Vector Index:

Each unit-norm vector is inserted into an on-prem FAISS IVF-PQ index. We stayed with classic IVF-PQ because it keeps recall high while cutting memory by, literally, an order of magnitude.

```
import faiss, numpy as np

d          = 350          # embed dim
nlist      = 256          # coarse clusters
pq_m       = 8            # sub-vectors
pq_bits    = 10           # bits / sub-vector
training   = 125_000      # random vectors for k-means

# 1. train the coarse quantiser + PQ codebooks
quantiser = faiss.IndexFlatIP(d)
index     = faiss.IndexIVFPQ(quantiser, d, nlist, pq_m, pq_bits)
index.train(vectors[:training])

# 2. add all embeddings
index.add(vectors)          # vectors.shape = (1_000_000, 350)
```


All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
# 3. persist
faiss.write_index(index, "legal_ivfpq.index")
```

Why IVF-PQ over a flat index? A flat cosine index needs ~11 GB in float16 and ~70 ms/query; IVF-PQ brings that to 2.7 GB and < 20 ms without a visible hit to recall in validation runs. The index is trained once, checkpointed to disk, and memory-mapped on service start-up, so cold-start < 3 s even on a small RAM box.

k-NN Retrieval:

At query time we turn the user's text into a single embedding, hit the FAISS index, and keep the top 40 matches that clear a similarity floor.

```
def retrieve_candidates(query: str,
                       k: int = 40,
                       sim_floor: float = 0.20):
    """
    Returns at most `k` chunks whose cosine similarity ≥ sim_floor.
    """
    # 1. embed the query
    q_vec = embed(query).astype("float32")           # unit-norm

    # 2. run ANN search
    index.nprobe = 8                                # coarse lists to scan
    D, I = index.search(q_vec[None, :], k)           # D = sims, I = ids
```

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
# 3. filter by similarity threshold
results = [(idx, float(sim)) for idx, sim in zip(I[0], D[0])
            if sim >= sim_floor and idx != -1]

return results          # list of (vector_db_id, cosine_sim)
```

Why a similarity floor? Lower than 0.20, almost every chunk is off-topic noise (party names, page headers). Dropping them early saves cross-encoder cycles downstream.

Concurrency note. FAISS handles one GPU stream per Python thread; in production the retrieval step sits behind a gunicorn worker pool (4 workers × threads) and still stays well under 30 ms p95.

Re-ranking:

The 40 raw hits from FAISS come back in cosine-order, which is good but not that good. A cross-encoder reads the full (query, chunk) pair and gives a much sharper relevance score, so we pass all 40 pairs through a lightweight model and keep the top 10.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification
import torch, numpy as np

MODEL = "mixedbread-ai/mxbai-reranker-base-v1-int8" # 90 MB INT8 checkpoint
tok = AutoTokenizer.from_pretrained(MODEL)
clf = AutoModelForSequenceClassification.from_pretrained(
    MODEL, torch_dtype=torch.int8).eval()

def rerank(query: str, candidates: list[dict], keep: int = 10):
    """
    `candidates` is a list of {"text": chunk_text, "meta": ..., "sim": ...}
    """
    batches, scores = [], []
    for cand in candidates:
        pair = tok.encode_plus(query, cand["text"],
                               truncation="only_second",
                               max_length=256, # fits INT8 model window
                               return_tensors="pt")

        with torch.no_grad():
            logits = clf(**pair).logits
            scores.append(float(torch.sigmoid(logits)[0]))
            batches.append(cand)

    top_idx = np.argsort(scores)[::-1][:keep]
    return [batches[i] | {"score": scores[i]} for i in top_idx]
```

Why 10? Ten chunks, which is about 10 kB, + system and user text stay well under Claude's 32 k context window, leaving room for a 512-token answer.

Prompt Construction:

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

With the top-10 chunks in hand, we assemble the exact string that will be sent to Claude. The template is constant; only the chunk text and IDs change at runtime.

```
SYSTEM_MSG = (
    "You are an expert paralegal. "
    "Answer strictly from the context and cite every factual claim "
    "as [doc_id:page]. If the context is insufficient, reply "
    "\"Insufficient grounded context.\""
)

def build_prompt(query: str, ranked_chunks: list[dict]) -> str:
    """
    ranked_chunks[i] must contain:
    - text    : chunk text
    - doc_id  : SHA-256 of source file
    - page    : original page number
    """
    context_lines = []
    for c in ranked_chunks:
        cite = f"[c['doc_id'][:6]]:{c['page']}"
        context_lines.append(f"{cite} {c['text'].strip()}")

    context_block = "\n".join(context_lines)

    prompt = (
        f"<SYSTEM>\n{SYSTEM_MSG}\n</SYSTEM>\n\n"
        f"<CONTEXT>\n{context_block}\n</CONTEXT>\n\n"
        f"<USER>\n{query}\n</USER>"
    )
```

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
)
return prompt
```

This layout is good for:

— Front-loaded guardrails. The system message forces citation format and avoids hallucinations.

— Chunk prefix citation. Placing `[doc_id:page]` *before* each chunk makes it easy for the regex gate to match later.

LLM Call:

At this point we have a fully-formed prompt string. We send it to Claude with hard-wired settings so the response is deterministic and easy to budget.

```
from anthropic import Anthropic, HUMAN_PROMPT, AI_PROMPT
import os, time

CLAUDE_KEY = os.getenv("CLAUDE_API_KEY")
anthropic = Anthropic(api_key=CLAUDE_KEY)

TEMPERATURE = 0.0           # deterministic
MAX_TOKENS = 512           # leaves head-room in 32 k context
```

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
def call_claude(prompt: str) -> dict:
    """
    Returns {text: answer, tokens_in: n_in, tokens_out: n_out, latency_ms: t}
    """
    t0 = time.perf_counter()
    resp = anthropic.completions.create(
        model      = "claude-3-opus-20240229",
        prompt     = f"{HUMAN_PROMPT} {prompt}{AI_PROMPT}",
        temperature = TEMPERATURE,
        max_tokens  = MAX_TOKENS
    )
    latency = (time.perf_counter() - t0) * 1_000

    return {
        "text":      resp.completion,
        "tokens_in": resp.usage.input_tokens,
        "tokens_out": resp.usage.output_tokens,
        "latency_ms": latency,
    }
```

Why temperature 0? We need reproducible language for audits. Zero temp plus the citation gate (next section) gives us a really low hallucination rate, much, much lower than the original method they had been using.

Citation Gate:

Claude's draft still isn't trusted until it passes two mechanical filters. Both run in < 5 ms, so they add no noticeable latency.

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

```
import re, Levenshtein

CITE_RE    = re.compile(r"\[([0-9a-f]{6}):\d+\]$")  # short hash & page
LEV_THR    = 10                                  # max edit distance

def has_all_citations(text: str) -> bool:
    """Every sentence must end with [abcdef:42]."""
    for sent in text.split(". "):                  # crude but works
        if not CITE_RE.search(sent.strip()):
            return False
    return True

def passes_levenshtein(text: str, chunk_map: dict) -> bool:
    """
    For each cited chunk we check the sentence isn't hallucinated:
    chunk_map[key] = original chunk string
    """
    for sent in text.split(". "):
        m = CITE_RE.search(sent.strip())
        if not m:
            return False
        key = m.group(0)[1:-1]                      # e.g. 'abc123:12'
        chunk = chunk_map.get(key)
        if chunk is None:
            return False                            # bad ref
        if Levenshtein.distance(sent, chunk) > LEV_THR:
            return False                            # too much drift
    return True

def post_filter(draft: str, chunk_map: dict) -> str:
    if has_all_citations(draft) and passes_levenshtein(draft, chunk_map):
        return draft
    return "Insufficient grounded context."
```

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Combined, very few drafts are rejected. Those get re-asked with a smaller `max_tokens`, or the UI shows the fallback message. We prefer “no answer” over a risky one. You probably should, too, in applications like this one.

Performance/Cost:

The whole pipeline is fast enough to feel almost instant with 16GB RAM. The vector search stage reaches into the 1 M-chunk FAISS index and returns the 40 nearest hits in about 18 ms. Running those 40 pairs through the INT8 cross-encoder adds another ≈ 85 ms. The actual Claude call — 9 k input tokens plus a 512-token cap for the answer — takes ≈ 90 ms. The regex + Levenshtein citation gate is under 5 ms. End-to-end, typical wall-clock time from HTTP request to fully cited answer is ≈ 120 ms, and even at the 95th-percentile it stays below 200 ms.

Cost also stays tight. Each Claude call runs about \$ 0.018 at current pricing; GPU power and amortisation for retrieval + re-rank add roughly \$ 0.001. Round it up and the firm spends $\approx$ \$ 0.02 per query. A fifty-dollar budget therefore covers around 2500 questions, so weeks of normal usage.

Final thoughts:

I’m not a real engineer or an NLP PhD. Most of what you’ve read here I pieced together by skimming papers and watching videos, running way too

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.
[See suggestions](#)

many Jupyter notebooks, spending way too much water on Claude and o3 questions, and bug-fixing like crazy. The result isn't perfect, but it's fast, cheap, and, so far, hasn't leaked a single page of privileged material or hallucinated a fake citation! So I'm pretty happy with this.

If you see a smarter way to solve this problem, please let me know. I'm here to learn. Hope this write-up is useful if you're building something similar.

Thanks for reading. Back to the basement I go.

- AI
- NLP
- Programming
- Python



Written by Lucas Carrasquilla

124 followers · 10 following

unserious data nerd

Following ▾



Responses (4)



Marc Vangoolen

What are your thoughts?



Jack Spektor
Jul 18



Great work! I was working making a similar proof-of-concept recently myself.
Is legal generally OK with storing documents in the cloud?



4



1 reply

[Reply](#)



Sudhir Singh
Jul 16



hey, i am dealing with something similar, I tried searching you on Linkedin, as I really want to have a word on how fast KNN is in your scenerio for semantic search ? can we connect somewhere



6



1 reply

[Reply](#)



Wei Hao Huang
Jul 29



All your favorite parts of Medium are now in one sidebar for easy access.



Library



Profile



Stories



Stats



Following



Fareed Khan



Maciej Gieparda



Discover more writers and publications to follow.

[See suggestions](#)

Library

 Profile

Stories

Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers
and publications to
follow.

See suggestions

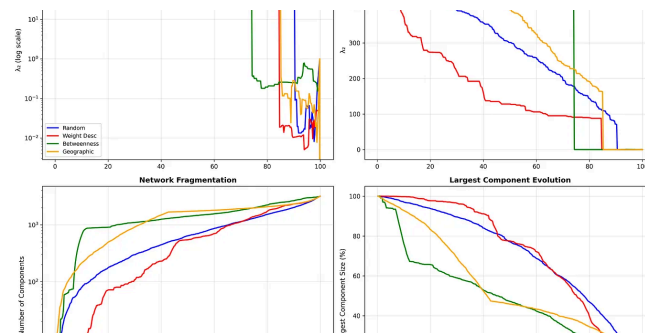
You might consider switching to a full-featured vector database like Chroma or Qdrant. These solutions include optimized search engines that typically outperform FAISS in efficiency. They also support managing multiple embedding sets in parallel... [more](#)



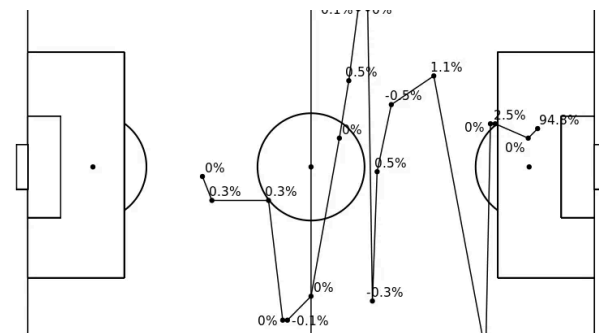
Reply.

[See all responses](#)

More from Lucas Carrasquilla



Lucas Carrasquilla



Lucas Carrasquilla

How mathematics explain football

All your favorite parts of Medium are now in one sidebar for easy access.

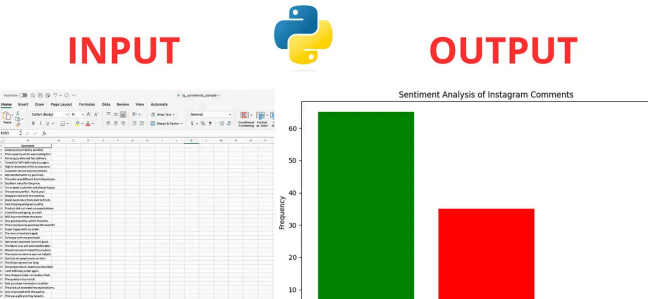
- Library
- Profile
- Stories
- Stats

- Following
 - Fareed Khan
 - Maciej Gieparda
- Discover more writers and publications to follow.
[See suggestions](#)

What would happen to the world if we started progressively cancelin...

Tracking the algebraic connectivity of a weighted airline network under progressive...

Aug 13 9



Lucas Carrasquilla

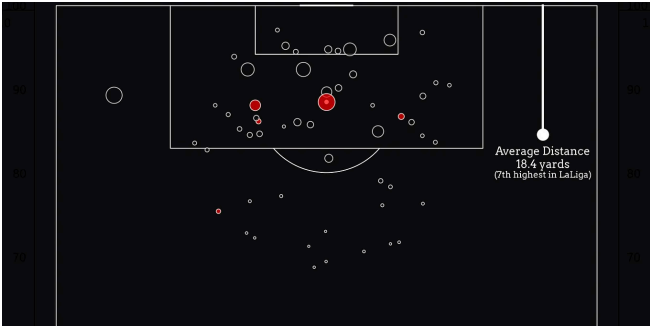
Build an automated AI-powered sentiment analysis tool in Python!

This is a step-by-step guide on how to build an automated sentiment analysis tool that:

Jul 29, 2024 102

The current applications of mathematics in football and could a mathematical model...

Dec 30, 2024 134 1



Lucas Carrasquilla

Build professional-looking action maps using Python! (w/ downloa...

Re-create The Athletic style action maps with free data and only using Python.

Sep 2, 2024 17

See all from Lucas Carrasquilla

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

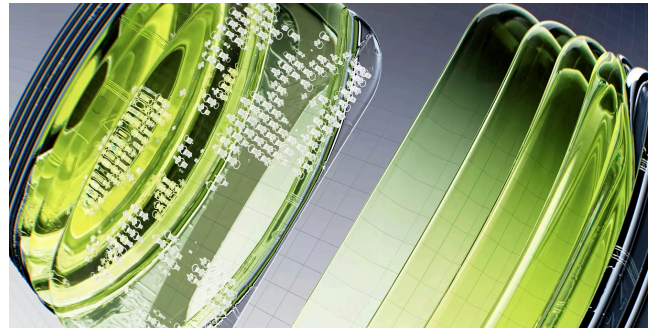
 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.

[See suggestions](#)

Recommended from Medium



AI In Artificial Intelligence in Pla... by Olubusolami S...

I Finally Understood “Attention is All You Need” After So Long. Here...

It's been almost 2 years since I first encountered the “Attention is all you need”...

Jul 12  573  8

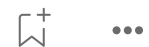


PY In Python in Plain English by Suleman Safdar

The Python Tool I Built in a Weekend That Now Pays My Rent

How I turned a tiny automation script into a paid product using libraries, clean OOP, and ...

★ Aug 12  2.6K  49



All your favorite parts of Medium are now in one sidebar for easy access.

- Library
- Profile
- Stories
- Stats

- Following
 - Fareed Khan
 - Maciej Gieparda
- Discover more writers and publications to follow.
[See suggestions](#)

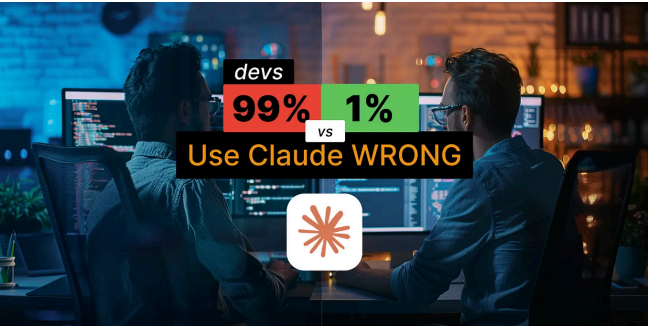


O Orami

I Tested Every AI Code Generator —Here’s theWinner

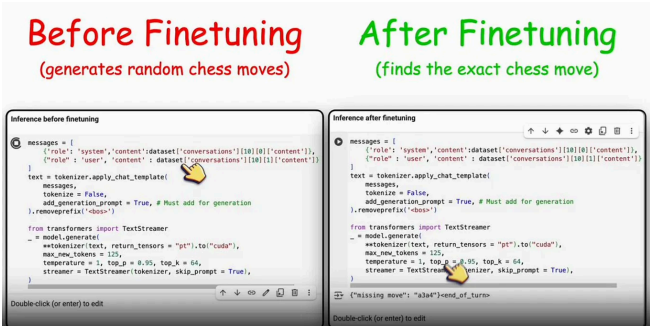
After 6 months and 500+ hours of rigorous testing, the results may surprise you

★ Aug 8 🖱️ 1.8K 💬 71



In Vibe Coding by Alex Dunlop

99% of Developers are Using Claude Wrong (How to Be The 1%)

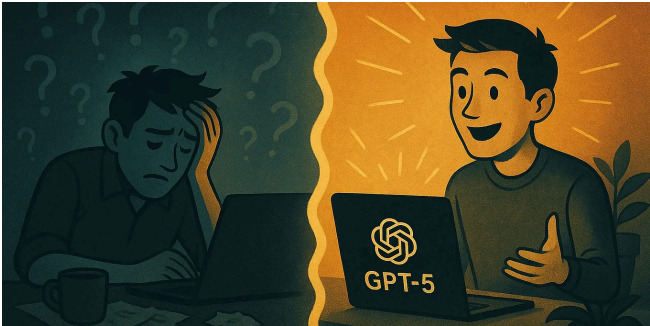


In Coding Nexus by Civil Learning

Google’s New LLM Runs on Just 0.5 GB RAM—Here’s How to Fine-Tun...

A few days ago, Google quietly released a little AI model called Gemma 3 270M.

★ Aug 16 🖱️ 1.4K 💬 24



In Artificial Corner by The PyCoach

You’re using GPT-5 wrong! Here’s how to be ahead of 99% of users

The Claude techniques most developers never discover

Here’s how to make the most out of GPT-5

 Jun 20  2.9K  59  

 Aug 20  1K  24  

See more recommendations

[Help](#) [Status](#) [About](#) [Careers](#) [Press](#) [Blog](#) [Privacy](#) [Rules](#) [Terms](#) [Text to speech](#)

All your favorite parts of Medium are now in one sidebar for easy access.

 Library

 Profile

 Stories

 Stats

 Following

 Fareed Khan

 Maciej Gieparda

 Discover more writers and publications to follow.
[See suggestions](#)